

Relatório Base de Dados
(Respostas aos exercícios)



Pedro Jorge N° 54878
Diogo Tavares N° 58049

1-

membro:

- Chave Primária: {IdMemb};
- Chave Candidatas:{IdMemb}
- Chaves Estrangeiras: Nenhuma

doce:

- Chave Primária: {Nome}
- Chave Candidatas: {Nome}
- Chaves Estrangeiras: Nenhuma

amigo:

- Chave Primária: {Membro1, Membro2}
- Chave Candidata: {Membro1, Membro2}
- Chaves Estrangeiras: {Membro1} ;{ Membro2}

criou:

- Chave Primária: {Membro, Doce}
- Chave Candidata:{Membro, Doce}
- Chaves Estrangeiras: {Membro}; {Doce}

fez:

- Chave Primária: {Membro, Doce}
- Chave Candidata: {Membro, Doce}
- Chaves Estrangeiras: {Membro}; {Doce}

ingredientes:

- Chave Primária: {Nome}
- Chave Candidata: {Nome}
- Chaves Estrangeiras: Nenhuma

temIngrediente:

- Chave Primária: {Doce, Ingrediente}
- Chave Candidata: {Doce, Ingrediente}
- Chaves Estrangeiras: {Doce}; {Ingrediente}

2-

```
CREATE TABLE membro (  
    Nome VARCHAR(100) PRIMARY KEY,  
    IdMemb INT PRIMARY KEY,  
    País VARCHAR(100),  
    DataNasc DATE
```

```
);
```

```
CREATE TABLE doce (  
    Nome VARCHAR(100) PRIMARY KEY,  
    Descrição TEXT,  
    Género VARCHAR(50)
```

```
);
```

```
CREATE TABLE amigo (  
    Membro1 INT,  
    Membro2 INT,  
    PRIMARY KEY (Membro1, Membro2),  
    FOREIGN KEY (Membro1) REFERENCES membro(IdMemb),  
    FOREIGN KEY (Membro2) REFERENCES membro(IdMemb)
```

```
);
```

```
CREATE TABLE criou (  
    Membro INT,  
    Doce VARCHAR(100),  
    PRIMARY KEY (Membro, Doce),  
    FOREIGN KEY (Membro) REFERENCES membro(IdMemb),  
    FOREIGN KEY (Doce) REFERENCES doce(Nome)
```

```
);
```

```

CREATE TABLE fez (
    Membro INT,
    Doce VARCHAR(100),
    Tempo INT,
    Aspeto INT,
    Sabor INT,
    PRIMARY KEY (Membro, Doce),
    FOREIGN KEY (Membro) REFERENCES membro(IdMemb),
    FOREIGN KEY (Doce) REFERENCES doce(Nome)
);

```

```

CREATE TABLE ingrediente (
    Nome VARCHAR(100) PRIMARY KEY,
    Custo DECIMAL(10, 2)
);

```

```

CREATE TABLE temIngrediente (
    Doce VARCHAR(100),
    Ingrediente VARCHAR(100),
    Quantidade INT,
    PRIMARY KEY (Doce, Ingrediente),
    FOREIGN KEY (Doce) REFERENCES doce(Nome),
    FOREIGN KEY (Ingrediente) REFERENCES ingrediente(Nome)
);

```

3

A)-

```

INSERT INTO membro (Nome, IdMemb, País, DataNasc) VALUES
    ('Ana Silva', 1, 'Portugal', '1990-05-20'),
    ('João Pereira', 2, 'Brasil', '1985-11-15'),
    ('Cláudia Santos', 3, 'Angola', '1992-01-30'),
    ('Rui Gomes', 4, 'Moçambique', '1988-08-10'),
    ('Sofia Almeida', 5, 'Espanha', '1995-12-25'),
    ('Miguel Costa', 6, 'França', '1987-01-14'),
    ('Carla Martins', 7, 'Itália', '1993-03-03'),
    ('Pedro Ferreira', 8, 'Reino Unido', '1980-07-21'),
    ('Tiago Ribeiro', 9, 'Espanha', '1994-06-30'),
    ('Mariana Lopes', 10, 'Alemanha', '1989-04-12');

```

B)-

```

INSERT INTO ingrediente (Nome, Custo) VALUES
    ('Açúcar', 1.00),
    ('Farinha', 0.50),
    ('Chocolate', 2.00),

```

('Ovos', 0.30),
('Natas', 1.50),
('Leite', 0.80),
('Água', 0.10),
('Maçã', 0.60),
('Manteiga', 1.20),
('Alfarroba', 1.50),
('Baunilha', 2.50),
('Canela', 0.40),
('Pimenta', 0.20);

C)-

INSERT INTO doce (Nome, Descrição, Género) VALUES

('Bolo de Chocolate', 'Um delicioso bolo de chocolate.', 'Regional'),
('Tarte de Maçã', 'Uma tarte clássica com maçãs.', 'Tradicional'),
('Pudim de Alfarroba', 'Pudim feito com alfarroba.', 'Regional'),
('Bolo de Baunilha', 'Bolo simples e saboroso de baunilha.', 'Regional'),
('Torta de Canela', 'Torta recheada com canela.', 'Tradicional'),
('Mousse de Natas', 'Mousse leve feita com natas.', 'Regional'),
('Arroz Doce', 'Um doce tradicional feito com arroz.', 'Tradicional'),
('Bolo de Pimenta', 'Bolo picante com pimenta.', 'Regional'),
('Bolo de Maçã', 'Bolo macio com pedaços de maçã.', 'Regional'),
('Torta de Chocolate', 'Torta rica em chocolate.', 'Tradicional');

INSERT INTO criou (Membro, Doce) VALUES

(1, 'Bolo de Chocolate'),
(2, 'Tarte de Maçã'),
(3, 'Pudim de Alfarroba'),
(4, 'Bolo de Baunilha'),
(5, 'Torta de Canela'),
(1, 'Mousse de Natas'),
(2, 'Arroz Doce'),
(3, 'Bolo de Pimenta'),
(4, 'Bolo de Maçã'),
(5, 'Torta de Chocolate');

INSERT INTO fez (Membro, Doce, Tempo, Aspeto, Sabor) VALUES

(1, 'Bolo de Chocolate', 30, 6, 7),
(2, 'Bolo de Chocolate', 30, 7, 8),
(1, 'Tarte de Maçã', 25, 6, 7),
(2, 'Tarte de Maçã', 25, 6, 7),
(3, 'Tarte de Maçã', 25, 5, 6),
(1, 'Pudim de Alfarroba', 20, 6, 7),
(2, 'Pudim de Alfarroba', 20, 7, 8),
(4, 'Bolo de Baunilha', 20, 7, 9),
(5, 'Torta de Canela', 20, 8, 7),
(1, 'Mousse de Natas', 15, 5, 8),
(2, 'Arroz Doce', 35, 5, 6),

(3, 'Bolo de Pimenta', 20, 5, 5),
(4, 'Bolo de Maçã', 15, 6, 7),
(5, 'Torta de Chocolate', 25, 5, 6),
(1, 'Bolo de Maçã', 15, 6, 7),
(2, 'Bolo de Maçã', 15, 6, 8),
(3, 'Bolo de Maçã', 15, 6, 6);

D)-

INSERT INTO amigo (Membro1, Membro2) VALUES (1, 2),

(2, 1),
(1, 3),
(3, 1),
(1, 4),
(4, 1),
(1, 5),
(5, 1),
(1, 6),
(6, 1),
(1, 7),
(7, 1),
(1, 8),
(8, 1),
(1, 9),
(9, 1),
(1, 10),
(10, 1),
(2, 3),
(3, 2),
(2, 4),
(4, 2),
(3, 5),
(5, 3),
(5, 7),
(7, 5),
(6, 8),
(8, 6),
(7, 9),
(9, 7),
(8, 10),
(10, 8);

E)-

Para responder a alinha 4a) foi preciso inserir os seguintes valor:

```
INSERT INTO doce (Nome, Descrição, Género) VALUES
('Torta de Chocolate e Canela', 'Doce tradicional espanhol de chocolate com
um toque de canela.', 'Regional');
INSERT INTO criou (Membro, Doce) VALUES
(5, 'Torta de Chocolate e Canela'),
(9, 'Torta de Chocolate e Canela');
INSERT INTO temIngrediente (Doce, Ingrediente, Quantidade) VALUES
('Torta de Chocolate e Canela', 'Chocolate', 2),
('Torta de Chocolate e Canela', 'Canela', 1);
```

Para responder a alinha 4b) foi preciso inserir os seguintes valor:

```
INSERT INTO membro (Nome, IdMemb, País, DataNasc) VALUES
('Joaquim José', 11, 'Portugal', '1982-09-15'),
('Pedro Carmo', 12, 'Índia', '1999-05-18'),
('Rodrigo Barreto', 13, 'Angola', '1990-06-19');
INSERT INTO amigo (Membro1, Membro2) VALUES
(11, 12),
(12, 11),
(11, 13),
(13, 11);
```

```
INSERT INTO criou (Membro, Doce) VALUES
(12, 'Torta de Chocolate'),
(13, 'Arroz Doce');
INSERT INTO fez (Membro, Doce, Tempo, Aspeto, Sabor) VALUES
(12, 'Torta de Chocolate', 3, 4, 5),
(13, 'Arroz Doce', 2, 5, 4);
```

Para responder a alínea 4c) não é preciso inserir mais informações na tabela.

Para responder a 4d) não é preciso inserir mais informação pois já existe o doce com natas que não teve 5 sabor

Para a linha 4 e) também não é preciso inserir mais informação.

Para a alínea 4f) também não é preciso inserir mais informação.

Para a alínea 4g) também não é preciso inserir mais informação.

Para a alínea 4h também não é preciso inserir mais informação.

Para a alínea 4i) também não é preciso inserir mais informação.

Para a alínea 4j) também não é preciso inserir mais informação.

Para a alínea 4k) também não é preciso inserir mais informação.

Para responder a alínea l) foi preciso inserir os seguintes valores:

```
INSERT INTO membro (Nome, IdMemb, País, DataNasc) VALUES  
( 'Carlos Abreu', 14, 'Portugal', '2004-05-01');
```

```
INSERT INTO amigo (Membro1, Membro2) VALUES  
(12, 14),  
(14, 12),  
(13, 14),  
(14, 13);
```

Para responder a alínea m) tive que adicionar as seguintes informações:

```
INSERT INTO fez (Membro, Doce, Tempo, Aspeto, Sabor) VALUES  
(12, 'Torta de Chocolate', 2, 5, 4),  
(13, 'Torta de Chocolate', 2, 5, 4),  
(12, 'Bolo de Baunilha', 4, 3, 5),  
(13, 'Bolo de Baunilha', 4, 3, 5);
```

Para a alínea 4n) também não é preciso inserir mais informação.

Para a alínea 4o) também não é preciso inserir mais informação.

4- a)

SQL:

```
SELECT DISTINCT m.Nome
```

```
FROM membro m
```

```
JOIN criou c ON m.IdMemb = c.Membro
```

```
JOIN doce d ON c.Doce = d.Nome
```

```
JOIN temIngrediente ti ON d.Nome = ti.Doce
```

```
WHERE m.País = 'Espanha'
```

```
AND d.Género = 'Regional'
```

```
AND ti.Ingrediente IN ('Chocolate', 'Canela')
```

```
GROUP BY m.Nome, d.Nome
```

```
HAVING COUNT(DISTINCT ti.Ingrediente) = 2;
```

Algebra Relacional:

- $\text{MembrosEspanhois} \leftarrow \sigma_{\text{membro.País=Espanha}}(\text{membro})$
- $\text{DocesRegionais} \leftarrow \sigma_{\text{doce.Género = Regional}}(\text{doce})$
- $\text{DocesComChocolate} \leftarrow \pi_{\text{Doce}} \sigma_{\text{Ingrediente =chocolate}}(\text{temIngrediente})$
- $\text{DocesComCanela} \leftarrow \pi_{\text{Doce}} \sigma_{\text{Ingrediente =canela}}(\text{temIngrediente})$
- $\text{DocesComChocolateECanela} \leftarrow \text{DocesComChocolate} \cap \text{DocesComCanela}$
- $\text{DocesRegionaisComChocolateECanela} \leftarrow \text{DocesRegionais} \cap \text{DocesComChocolateECanela}$
- $\text{MembrosEspanhoisCriaramDoces} \leftarrow \sigma_{\text{MembrosEspanhois.IdMemb=criou.Membro}}(\text{criou} \bowtie \text{MembrosEspanhois})$
- $\text{Resultado} \leftarrow \pi_{\text{Nome}}(\sigma_{\text{MembrosEspanhois.IdMemb=criou.Membro}}(\text{MembrosEspanhois} \text{ CriaramDoces} \cap \text{DocesRegionaisComChocolateECanela}))$

b)

SQL:

```
SELECT d.Nome
FROM doce d
JOIN criou c ON d.Nome = c.Doce
JOIN amigo a ON c.Membro = a.Membro2
JOIN fez f ON f.Membro = a.Membro2 AND f.Doce = d.Nome
WHERE a.Membro1 = 11
      AND d.Género = 'Tradicional'
      AND f.Aspeto = 5;
```

Algebra relacional:

- $\text{IdJoaquim} \leftarrow \pi_{\text{IdMemb}} \sigma_{\text{nome}='Joaquim José'}(\text{membro})$
- $\text{IdAmigosJoaquim} \leftarrow \pi_{\text{Membro2}} \sigma_{\text{Membro1} = \text{IdJoaquim}}(\text{amigo}) \cup \pi_{\text{Membro1}} \sigma_{\text{Membro2} = \text{IdJoaquim}}(\text{amigo})$
- $\text{DocesAmigosJoaquim} \leftarrow \pi_{\text{doce}} \sigma_{\text{criou.membro} = \text{IdAmigosJoaquim}}(\text{criou})$
- $\text{DocesTradicionais} \leftarrow \pi_{\text{nome}} \sigma_{\text{Género} = 'Tradicional'}(\text{doce})$
- $\text{DocesTradicionaisCriadosPorAmigos} \leftarrow \text{DocesAmigosJoaquim} \cap \text{DocesTradicionais}$
- $\text{DocesComAspeto5} \leftarrow \pi_{\text{Doce}} (\sigma_{\text{Aspeto} \geq 5}(\text{fez}))$
- $\text{Resultado} \leftarrow \text{DocesTradicionaisCriadosPorAmigos} \cap \text{DocesComAspeto5}$

c)

SQL:

```
SELECT m.Nome
FROM membro m
WHERE m.IdMemb IN (
    SELECT f.Membro
    FROM fez f
    JOIN criou c ON f.Doce = c.Doce
    WHERE f.Membro IN (
        SELECT amigo.Membro2
        FROM amigo
        WHERE amigo.Membro1 = m.IdMemb
    )
    GROUP BY f.Membro
)
AND NOT EXISTS (
    SELECT *
    FROM fez f2
    JOIN criou c2 ON f2.Doce = c2.Doce
    WHERE f2.Membro = m.IdMemb
    AND c2.Membro <> m.IdMemb
);
```

Algebra Relacional:

- $\text{MembroDoces} \leftarrow \pi_{\text{Membro}, \text{Doce}}(\text{fez})$
- $\text{Amigos} \leftarrow \pi_{\text{Membro1}, \text{Membro2}}(\text{amigo}) \cup \pi_{\text{Membro2}, \text{Membro1}}(\text{amigo})$
- $\text{DocesCriadosPorAmigos} \leftarrow \pi_{\text{Amigos.Membro1}, \text{criou.Doce}}(\sigma_{\text{Amigos.Membro2}=\text{criou.Membro}}(\text{Amigos} \times \text{criou}))$
- $\text{DocesTradicionais} \leftarrow \sigma_{\text{doce.Género} = \text{'Tradisional'}}(\text{doce})$
- $\text{DocesTradicionaisCriadosPorAmigos} \leftarrow \sigma_{\text{DocesCriadosPorAmigos.Doce}=\text{DocesTradicionais.Doce}}(\text{DocesCriadosPorAmigos} \times \text{DocesTradicionais})$
- $\text{DocesComAspeto5} \leftarrow \sigma_{\text{fez.Aspeto} \geq 5}(\text{fez})$
- $\text{Resultado} \leftarrow \pi_{\text{DocesTradicionaisCriadosPorAmigos.Doce}}(\sigma_{\text{DocesTradicionaisCriadosPorAmigos.Doce}=\text{DocesComAspeto5.Doce}}(\text{DocesTradicionaisCriadosPorAmigos} \times \text{DocesComAspeto5}))$

d)

SQL:

```
SELECT d.Nome
FROM doce d
JOIN temIngrediente ti ON d.Nome = ti.Doce
JOIN fez f ON d.Nome = f.Doce
WHERE ti.Ingrediente = 'Natas'
AND f.Sabor < 5;
```

Algebra Relacional:

- $\text{DocesComNatas} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Ingrediente}='Natas'}(\text{temIngrediente}))$
- $\text{DocesComSabor5} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Sabor}=5}(\text{fez}))$
- $\text{Resultado} \leftarrow \text{DocesComNatas} - \text{DocesComSabor5}$

e)

SQL:

```
SELECT DISTINCT m.Nome
FROM membro m
JOIN criou c ON m.IdMemb = c.Membro
JOIN temIngrediente ti ON c.Doce = ti.Doce
WHERE ti.Ingrediente IN ('Canela', 'Leite')
GROUP BY m.Nome
HAVING COUNT(DISTINCT ti.Ingrediente) = 2;
```

Algebra Relacional:

- $\text{DocesComCanela} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Ingrediente}='Canela'}(\text{temIngrediente}))$
- $\text{DocesComLeite} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Ingrediente}='Leite'}(\text{temIngrediente}))$
- $\text{DocesComCanelaELeite} \leftarrow \text{DocesComCanela} \cap \text{DocesComLeite}$
- $\text{Resultado} \leftarrow \pi_{\text{Membro}}(\sigma_{\text{criou.Doce}=\text{DocesComCanelaELeite.Doce}}(\text{criou} \bowtie \text{DocesComCanelaELeite}))$

f)

SQL:

```
SELECT DISTINCT d.Nome
FROM doce d
JOIN temIngrediente ti ON d.Nome = ti.Doce
WHERE ti.Ingrediente IN ('Maçã', 'Baunilha');
```

Algebra relacional:

- $\text{DocesComMaçã} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Ingrediente}='Maçã'}(\text{temIngrediente}))$
- $\text{DocesComBaunilha} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Ingrediente}='Baunilha'}(\text{temIngrediente}))$
- $\text{Resultado} \leftarrow \text{DocesComMaçã} \cup \text{DocesComBaunilha}$

g)

SQL:

```
SELECT SUM(i.Custo * ti.Quantidade / CASE
    WHEN ti.Ingrediente IN ('Arroz', 'Açúcar', 'Leite') THEN 100 --
consideramos 100 gramas/ml para o cálculo
    ELSE 1
END) AS CustoTotal
FROM temIngrediente ti
JOIN ingrediente i ON ti.Ingrediente = i.Nome
WHERE ti.Doce = 'Arroz Doce';
```

Algebra Relacional:

$\text{IngredientesArrozDoce} \leftarrow \sigma_{\text{Doce}='Arroz Doce'}(\text{temIngrediente})$
 $\text{IngredientesComCusto} \leftarrow \sigma_{\text{IngredientesArrozDoce.Ingrediente}=\text{ingrediente.Nome}}(\text{IngredientesArrozDoce} \bowtie \text{ingrediente})$

h)

SQL:

```
SELECT d.Nome AS Doce, SUM(i.Custo * ti.Quantidade) AS  
Custo_Total  
FROM doce d  
JOIN temIngrediente ti ON d.Nome = ti.Doce  
JOIN ingrediente i ON ti.Ingrediente = i.Nome  
GROUP BY d.Nome;
```

Algebra Relacional:

- $\text{IngredientesDoce} \leftarrow \sigma_{\text{temIngrediente.Ingrediente}=\text{ingrediente.Nome}}(\text{temIngrediente} \bowtie \text{ingrediente})$
- $\text{CustoPorIngrediente} \leftarrow \pi_{\text{Doce, Ingrediente, (Quantidade} \times \text{Custo)}} \text{as } \text{CustoIngrediente}(\text{IngredientesDoce})$
- $\text{Doce } \mathcal{G}_{\text{sum}(\text{CustoIngrediente})}(\text{CustoPorIngrediente})$

i)

SQL:

```
SELECT m.Nome, COUNT(c.Doce)  
FROM membro m  
LEFT JOIN criou c ON m.IdMemb = c.Membro  
GROUP BY m.Nome;
```

(algebra relacional na próxima página)

Algebra Relacional:

- $t_j \leftarrow \sigma_{\text{membro.IdMemb}=\text{criou.Membro}}(\text{criou} \bowtie \text{membro})$
- $\text{Resultado} \leftarrow \text{Nome } \mathcal{G}_{\text{count}(\text{doce})}(t_j)$

j)

SQL:

```
SELECT m.Nome, COUNT(c.Doce)
FROM membro m
JOIN criou c ON m.IdMemb = c.Membro
GROUP BY m.Nome
ORDER BY COUNT(c.Doce) DESC
LIMIT 1;
```

Algebra Relacional:

- $t_j \leftarrow \sigma_{\text{membro.IdMemb}=\text{criou.Membro}}(\text{criou} \bowtie \text{membro})$
- $\text{contagem} \leftarrow \text{Nome } \mathcal{G}_{\text{count}(\text{doce})}(t_j)$
- $\text{Max} \leftarrow \mathcal{G}_{\text{max}(\text{Doce})}(\text{contagem})$
- $\text{Resultado} \leftarrow$

k)

SQL:

```
WITH Custo_Doces AS (  
    SELECT doce.Nome AS Nome_Doce, SUM(ingrediente.Custo *  
    temIngrediente.Quantidade) AS Custo_Total  
    FROM doce  
    JOIN temIngrediente ON doce.Nome = temIngrediente.Doce  
    JOIN ingrediente ON temIngrediente.Ingrediente = ingrediente.Nome  
    GROUP BY doce.Nome  
)  
Doce_Mais_Caro AS (  
    SELECT Nome_Doce  
    FROM Custo_Doces  
    WHERE Custo_Total = (SELECT MAX(Custo_Total) FROM  
    Custo_Doces)  
)
```

Algebra Relacional:

- $\text{IngredientesDoce} \leftarrow \sigma_{\text{temIngrediente.Ingrediente}=\text{ingrediente.Nome}}(\text{temIngrediente} \bowtie \text{ingrediente})$
- $\text{CustoPorIngrediente} \leftarrow \pi_{\text{Doce, Ingrediente, (Quantidade} \times \text{Custo)}} \text{as } \pi_{\text{CustoIngrediente}}(\text{IngredientesDoce})$
- $\text{DoceMaisCaroEPreço} \leftarrow \text{Doce } \mathcal{G}_{\max(\text{CustoIngrediente})}(\text{CustoPorIngrediente})$
- $\text{NomeMembros} \leftarrow \pi_{\text{Nome}}(\text{membro})$
- $\text{IdsFizeramDoceMaisCaro} \leftarrow \pi_{\text{Membro}} \sigma_{\text{faz.doce}=\text{doce}}(\text{faz} \bowtie \text{DoceMaisCaroEPreço})$
- $\text{NomeMembros} \leftarrow \pi_{\text{Nome}} \sigma_{\text{IdMembro}=\text{membro}}(\text{membro} \bowtie \text{IdsFizeramDoceMaisCaro})$

l)

SQL:

```
SELECT m1.Nome
FROM amigo a1
JOIN amigo a2 ON a1.Membro2 = a2.Membro2
JOIN membro m1 ON a2.Membro1 = m1.IdMemb
WHERE a1.Membro1 = (SELECT IdMemb FROM membro WHERE
Nome = 'Joaquim José')
GROUP BY m1.Nome
HAVING COUNT(DISTINCT a2.Membro1) = (SELECT COUNT(*) FROM
amigo WHERE Membro1 = (SELECT IdMemb FROM membro WHERE
Nome = 'Joaquim José'));
```

Algebra Relacional:

- $\text{IdJoaquim} \leftarrow \pi_{\text{IdMemb}} \sigma_{\text{nome}='Joaquim José'}(\text{membro})$
- $\text{IdAmigosJoaquim} \leftarrow \pi_{\text{Membro2}} \sigma_{\text{Membro1} = \text{IdJoaquim}}(\text{amigo}) \cup \pi_{\text{Membro1}} \sigma_{\text{Membro2} = \text{IdJoaquim}}(\text{amigo})$
- $\text{AmigosDosAmigos1} \leftarrow \pi_{\text{Membro2}} (\sigma_{\text{Membro1} = \text{id.IdAmigosJoaquim}}(\text{amigo} \bowtie \text{IdAmigosJoaquim}))$
- $\text{AmigosDosAmigos2} \leftarrow \pi_{\text{Membro1}} (\sigma_{\text{Membro2} = \text{id.IdAmigosJoaquim}}(\text{amigo} \bowtie \text{IdAmigosJoaquim}))$
- $\text{IdAmigosDosAmigos} \leftarrow \text{AmigosDosAmigos1} \cup \text{AmigosDosAmigos2}$
- $\text{NomesDosAmigosDosAmigos} \leftarrow \pi_{\text{Nome}} \sigma_{\text{Membro} = \text{IdMemb}}(\text{IdAmigosDosAmigos} \bowtie \text{membro})$

M)
 SQL:
 SELECT Membro
 FROM fez
 WHERE Doce IN (
 SELECT Doce
 FROM doce
 WHERE Ingrediente = 'Baunilha'
)
 GROUP BY Membro
 HAVING COUNT(DISTINCT Doce) = (
 SELECT COUNT(DISTINCT Doce)
 FROM doce
 WHERE Ingrediente = 'Baunilha'

Algebra Relacional:

- $\text{DocesComBaunilha} \leftarrow \pi_{\text{Doce}}(\sigma_{\text{Ingrediente}='Baunilha'}(\text{temIngrediente}))$
- $\text{FezCBaunilha} \leftarrow \pi_{\text{Membro}, \text{Doce}} \sigma_{\text{fez.Doce}=\text{DocesComBaunilha.Doce}}(\text{fez} \bowtie \text{DocesComBaunilha})$
- $\text{MembrosEDocesFeitos} \leftarrow \pi_{\text{Membro}, \text{Doce}}(\text{fez})$
- $\text{MembrosEDocesSemBaunilha} \leftarrow \text{MembrosEDocesFeitos} - \text{MembrosEDocesComBaunilha}$
- $\text{IdSóBaunilha} \leftarrow \pi_{\text{Membro}}(\text{MembrosEDocesFeitos} - \text{MembrosEDocesSemBaunilha})$

n)

SQL:

```
SELECT DISTINCT Doce
FROM fez
WHERE Doce NOT IN (
    SELECT Doce
    FROM fez
    WHERE Tempo <> 1
```

Algebra Relacional:

```
TodosOsDoces  $\leftarrow \pi_{Doce}(fez)$ 
DocesComTempoDiferenteDe1  $\leftarrow \pi_{Doce}(\sigma_{Tempo=1}(fez))$ 
DocesSempreTempo1  $\leftarrow TodosOsDoces - DocesComTempoDiferenteDe1$ 
```

o)

SQL:

```
SELECT Nome
FROM doce
WHERE Custo = (SELECT MIN(Custo) FROM doce)
OR Tempo = (SELECT MIN(Tempo) FROM doce);
```

Algebra Relacional:

- $IngredientesDoce \leftarrow \sigma_{temIngrediente.Ingrediente=ingrediente.Nome}(temIngrediente \bowtie ingrediente)$
- $CustoPorIngrediente \leftarrow \pi_{Doce, Ingrediente, (Quantidade \times Custo)} as$
 $CustoIngrediente(IngredientesDoce)$
- $CustoDoce \leftarrow_{Doce} \mathcal{G}_{custo} as sum(CustoIngrediente)(CustoPorIngrediente)$
- $MenorCusto \leftarrow \pi_{min(Custo)}(CustoDoce)$
- $DocesMaisBaratos \leftarrow \pi_{Doce}(\sigma_{Custo=MenorCusto}(CustoDoce))$
- $MenorTempo \leftarrow \pi_{min(Tempo)}(fez)$
- $DocesMaisRapidos \leftarrow \pi_{Doce}(\sigma_{Tempo=MenorTempo}(fez))$
- $DocesMaisRapidosEBaratos \leftarrow DocesMaisRapidos \cup DocesMaisBaratos$